Pseudocode

Dynamic Programming

Longest Common Subsequence

```
function LCS(X, Y):
    m = length(X)
    n = length(Y)

    create dp[0..m][0..n] initialized to 0

    for i = 1 to m:
        for j = 1 to n:
            if X[i-1] == Y[j-1]:
                dp[i][j] = 1 + dp[i-1][j-1]
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])

    return dp[m][n]
```

0/1 Knapsack

```
function KNAPSACK_01(values, weights, max_weight):
    n = length(values)
    create dp[0..n][0..max_weight] initialized to 0

    for i = 1 to n:
        for w = 0 to max_weight:
            if weights[i-1] > w:
                dp[i][w] = dp[i-1][w]
            else:
                dp[i][w] = max(
                    dp[i-1][w],
                    values[i-1] + dp[i-1][w - weights[i-1]]
                )

    return dp[n][max_weight]
```

Unbounded Knapsack

```
function UNBOUNDED_KNAPSACK(values, weights, max_weight):
    n = length(values)
    create dp[0..max_weight] initialized to 0

    for w = 0 to max_weight:
        for i = 0 to n-1:
            if weights[i] <= w:
                dp[w] = max(dp[w], dp[w - weights[i]] + values[i])

    return dp[max_weight]
```

Rod Cutting

```
function ROD_CUTTING(prices):
    n = length(prices)
    create dp[0..n] initialized to 0

    dp[0] = 0

    for i = 1 to n:
        max_val = 0
        for j = 1 to i:
            max_val = max(max_val, prices[j-1] + dp[i-j])
        dp[i] = max_val

    return dp[n]
```

Maximum Square of Zeros

```
function MAX_ZERO_SQUARE(matrix):
    rows = row_count(matrix)
    cols = column_count(matrix)

    create dp[0..rows-1][0..cols-1] initialized to 0
    max_size = 0

    for i = 0 to rows-1:
        for j = 0 to cols-1:
            if matrix[i][j] == 0:
                if i == 0 or j == 0:
                    dp[i][j] = 1
                else:
                    dp[i][j] = 1 + min(dp[i-1][j-1], dp[i-1][j], dp[i][j-1])
```

```
                max_size = max(max_size, dp[i][j])
            else:
                dp[i][j] = 0

    return max_size
```

Coin Row

```
function COIN_ROW(coins):
    n = length(coins)
    create dp[0..n] initialized to 0

    dp[0] = 0
    dp[1] = coins[0]

    for i = 2 to n:
        dp[i] = max(dp[i-1], dp[i-2] + coins[i-1])

    return dp[n]
```

Brute Force

Breadth-First Approach

ALGORITHM *BFS*(G)

```
//Implements a breadth-first search traversal of a given graph
//Input: Graph G = ⟨V, E⟩
//Output: Graph G with its vertices marked with consecutive integers
//in the order they have been visited by the BFS traversal
mark each vertex in V with 0 as a mark of being "unvisited"
count ← 0
for each vertex v in V do
    if v is marked with 0
      bfs(v)

bfs(v)
//visits all the unvisited vertices connected to vertex v by a path
//and assigns them the numbers in the order they are visited
//via global variable count
count ← count + 1;  mark v with count and initialize a queue with v
while the queue is not empty do
    for each vertex w in V adjacent to the front vertex do
        if w is marked with 0
            count ← count + 1;  mark w with count
            add w to the queue
    remove the front vertex from the queue
```

Depth-First Approach

ALGORITHM *DFS*(G)

//Implements a depth-first search traversal of a given graph
//Input: Graph $G = \langle V, E \rangle$
//Output: Graph G with its vertices marked with consecutive integers
//in the order they've been first encountered by the DFS traversal
mark each vertex in V with 0 as a mark of being "unvisited"
$count \leftarrow 0$
for each vertex v in V **do**
    if v is marked with 0
        dfs(v)

dfs(v)
//visits recursively all the unvisited vertices connected to vertex v by a path
//and numbers them in the order they are encountered
//via global variable *count*
$count \leftarrow count + 1$; mark v with *count*
for each vertex w in V adjacent to v **do**
    if w is marked with 0
        dfs(w)

Job Assignment Problem

```
BRUTE_FORCE_JOB_ASSIGNMENT(cost[1..n][1..n])
    minCost ← ∞
    bestAssignment ← null

    for each permutation P of {1,2,…,n} do
        totalCost ← 0
        for i ← 1 to n do
            totalCost ← totalCost + cost[i][P[i]]
        end for

        if totalCost < minCost then
            minCost ← totalCost
            bestAssignment ← P
        end if
    end for

    return bestAssignment, minCost
```

Nearest Points

```
BRUTE_FORCE_CLOSEST_PAIR(points[1..n])
    minDist ← ∞
    closestPair ← null

    for i ← 1 to n − 1 do
        for j ← i + 1 to n do
            d ← distance(points[i], points[j])
            if d < minDist then
                minDist ← d
                closestPair ← (points[i], points[j])
            end if
        end for
    end for

    return closestPair, minDist
```

Polynomial Evaluation

```
POLYNOMIAL_EVALUATION(coeff[0..n], x)
    value ← 0

    for i ← 0 to n do
        term ← coeff[i]
        for j ← 1 to i do
            term ← term × x
        end for
        value ← value + term
    end for

    return value
```

Convex Hull

```
BRUTE_FORCE_CONVEX_HULL(points[1..n])
    hull ← empty set

    for i ← 1 to n do
        for j ← i + 1 to n do
            left ← 0
            right ← 0

            for k ← 1 to n do
                if k ≠ i and k ≠ j then
                    pos ← orientation(points[i], points[j], points[k])
```

```
                    if pos > 0 then
                        left ← left + 1
                    else if pos < 0 then
                        right ← right + 1
                    end if
                end if
            end for

            if left = 0 or right = 0 then
                add points[i] and points[j] to hull
            end if
        end for
    end for

    return hull
```

Knapsack 0/1

```
BRUTE_FORCE_KNAPSACK(weights[1..n], values[1..n], W)
    maxValue ← 0

    for each subset S of {1,2,…,n} do
        totalWeight ← 0
        totalValue ← 0

        for each item i in S do
            totalWeight ← totalWeight + weights[i]
            totalValue ← totalValue + values[i]
        end for

        if totalWeight ≤ W and totalValue > maxValue then
            maxValue ← totalValue
        end if
    end for

    return maxValue
```

Matching Substring

```
MATCHING_SUBSTRING(P[0..n], T[0..m])
    for i = 0 to n - m:
        j = 0

        while T[i + j] == P[j] and j < m:
```

```
            j++

        if m == j return i
    return -1
```

Sorting Algorithms

Bubble Sort

```
BUBBLE_SORT(A[1..n])
    for i ← 1 to n − 1 do
        for j ← 1 to n − i do
            if A[j] > A[j + 1] then
                swap A[j] and A[j + 1]
            end if
        end for
    end for
```

Selection Sort

```
SELECTION_SORT(A[1..n])
    for i ← 1 to n − 1 do
        min ← i
        for j ← i + 1 to n do
            if A[j] < A[min] then
                min ← j
            end if
        end for
        swap A[i] and A[min]
    end for
```

Insertion Sort

```
INSERTION_SORT(A[1..n])
    for i ← 2 to n do
        key ← A[i]
        j ← i − 1
        while j ≥ 1 and A[j] > key do
            A[j + 1] ← A[j]
            j ← j − 1
        end while
        A[j + 1] ← key
    end for
```

Quicksort

```
QUICKSORT(A, low, high)
    if low < high then
        p ← PARTITION(A, low, high)
        QUICKSORT(A, low, p − 1)
        QUICKSORT(A, p + 1, high)
    end if
```

```
PARTITION(A, low, high)
    pivot ← A[high]
    i ← low − 1

    for j ← low to high − 1 do
        if A[j] ≤ pivot then
            i ← i + 1
            swap A[i] and A[j]
        end if
    end for

    swap A[i + 1] and A[high]
    return i + 1
```

Merge Sort

```
MERGE_SORT(A, left, right)
    if left < right then
        mid ← ⌊(left + right) / 2⌋
        MERGE_SORT(A, left, mid)
        MERGE_SORT(A, mid + 1, right)
        MERGE(A, left, mid, right)
    end if
```

```
MERGE(A, left, mid, right)
    n1 ← mid − left + 1
    n2 ← right − mid

    create arrays L[1..n1], R[1..n2]

    for i ← 1 to n1 do
        L[i] ← A[left + i − 1]
    end for
```

```
    for j ← 1 to n2 do
        R[j] ← A[mid + j]
    end for

    i ← 1, j ← 1, k ← left

    while i ≤ n1 and j ≤ n2 do
        if L[i] ≤ R[j] then
            A[k] ← L[i]
            i ← i + 1
        else
            A[k] ← R[j]
            j ← j + 1
        end if
        k ← k + 1
    end while

    while i ≤ n1 do
        A[k] ← L[i]
        i ← i + 1
        k ← k + 1
    end while

    while j ≤ n2 do
        A[k] ← R[j]
        j ← j + 1
        k ← k + 1
    end while
```

Greedy Approach

Prim's Algorithm

```
PRIM(G, w, r):
    for each vertex v in V:
        key[v] ← ∞
        parent[v] ← NIL

    key[r] ← 0
    Q ← all vertices in V (min-priority queue ordered by key)

    while Q is not empty:
        u ← EXTRACT-MIN(Q)

        for each vertex v adjacent to u:
```

```
            if v ∈ Q and w(u, v) < key[v]:
                parent[v] ← u
                key[v] ← w(u, v)
                DECREASE-KEY(Q, v, key[v])

    return parent
```

Kruskal's Algorithm

```
KRUSKAL(G = (V, E), w):
    A ← ∅                      // set of MST edges

    for each vertex v in V:
        MAKE-SET(v)            // initialize disjoint sets

    sort E in nondecreasing order of weight w

    for each edge (u, v) in E:
        if FIND-SET(u) ≠ FIND-SET(v):
            A ← A U {(u, v)}
            UNION(u, v)

    return A
```

From Exercises

- *All its DFS forests (for traversals starting at different vertices) will have the same number of trees
  - A DFS forest has **one tree per connected component**.
  - The graph you gave is **connected** (there is a path between every pair of vertices).
  - No matter which vertex you start DFS from, DFS will eventually visit **all vertices**.
  - DFS will always produce **one tree**
  - So **all DFS forests have the same number of trees**
    ✓ Answer: TRUE
- *All its DFS forests will have the same number of tree edges and the same number of back edges?*
  - Let:
    - `n = |V|` = number of vertices
    - `m = |E|` = number of edges
  - For any DFS on an **undirected connected graph**:
    1. **Tree edges**

        - DFS tree always has:
        - `n − 1 tree edges`
    2. **Back edges**
        - Every remaining edge is a back edge
        - Number of back edges:
            - `m − (n − 1)`
    - These values depend **only on the graph**, **not** on:
        - starting vertex
        - DFS order
    - So, this is **TRUE** too.
- In the standard DP formulation, we have a table where:
    - Rows represent items (indexed 0 to n)
    - Columns represent capacities (indexed 0 to W)
    - Entry `dp[i][w]` = maximum value achievable using items 0 through i with capacity w
    - **a. Values in a row (nondecreasing)?**
        - **True.**
        - For a fixed set of items (fixed row i), as we increase the capacity (move right across columns), we have more room available. The maximum value we can achieve can only stay the same or increase—it cannot decrease. With more capacity, we can at least achieve what we did with less capacity, and possibly do better.
        - Formally: `dp[i][w] ≤ dp[i][w+1]` for all valid w.
    - **b. Values in a column (nondecreasing)?**
        - **True.**
        - For a fixed capacity (fixed column w), as we consider more items (move down the rows), we have more items to choose from. The maximum value can only stay the same or increase—it cannot decrease. With more items available, we can at least match our previous best selection, and possibly improve it.
        - Formally: `dp[i][w] ≤ dp[i+1][w]` for all valid i.
        - **Both statements are true** because the knapsack DP table is monotonic in both dimensions—adding more capacity or more items to choose from cannot make our optimal solution worse.
- **Kruskal's algorithm:** ✅ Works with negative edge weights.
- **Prim's algorithm:** ✅ Works with negative edge weights.
    - **Reason:** Both algorithms always choose edges with the smallest weight that do not form a cycle (Kruskal) or connect to the growing MST (Prim). The sign of the edge weight does not affect this selection process, so negative edges are handled correctly.

- **a. If eee is a minimum-weight edge in a connected weighted graph, it must be among edges of at least one minimum spanning tree of the graph. ✅ True**
  - Reason: By the **cut property** of MSTs, the minimum-weight edge crossing any cut belongs to **some** MST.
- **b. If eee is a minimum-weight edge in a connected weighted graph, it must be among edges of** each **minimum spanning tree of the graph. ❌ False**
  - Reason: If there are multiple edges with the same minimum weight across different cuts, eee might appear in some MSTs but not in others.
- **c. If edge weights of a connected weighted graph are all distinct, the graph must have exactly one minimum spanning tree. ✅ True**
  - Reason: Distinct weights prevent ties. The MST is **unique** because the cut property always selects a single minimum-weight edge.
- **d. If edge weights of a connected weighted graph are not all distinct, the graph must have more than one minimum spanning tree. ❌ False**
  - Reason: Non-distinct weights **may allow multiple MSTs**, but it's **not guaranteed**. There could still be a unique MST even with repeated weights.

Algorithm Complexity Reference Table

Dynamic Programming Algorithms

Algorithm	Time Complexity	Space Complexity	Notes
Longest Common Subsequence (LCS)	O(m·n)	O(m·n)	m = length of first string, n = length of second string
Knapsack 0/1	O(n·W)	O(n·W)	n = number of items, W = maximum weight capacity
Unbounded Knapsack	O(n·W)	O(W)	Uses 1D array for space optimization
Rod Cutting	$O(n^2)$	O(n)	n = length of rod
Max Square of Zeros	O(rows·cols)	O(rows·cols)	Finds largest square submatrix of zeros
Coin Row	O(n)	O(n)	n = number of coins; can be optimized to O(1) space

Brute Force Algorithms

Algorithm	Time Complexity	Space Complexity	Notes
Job Assignment	$O(n! \cdot n)$	$O(n)$	Tries all n! permutations, evaluates each in O(n)
Nearest Points (Closest Pair)	$O(n^2)$	$O(1)$	Compares all pairs of points
Polynomial Evaluation	$O(n^2)$	$O(1)$	Naive method without Horner's rule
Convex Hull	$O(n^3)$	$O(n)$	Checks each edge against all points
Knapsack 0/1 (Brute Force)	$O(2^n \cdot n)$	$O(n)$	Tries all 2^n subsets
Matching Substring	$O((n-m) \cdot m)$	$O(1)$	n = text length, m = pattern length

Sorting Algorithms

Algorithm	Time Complexity (Best)	Time Complexity (Average)	Time Complexity (Worst)	Space Complexity	Stable?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quicksort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No

Greedy/Graph Algorithms

Algorithm	Time Complexity	Space Complexity	Notes
Prim's Algorithm	$O((V+E) \cdot \log V)$	$O(V)$	With binary heap; $O(V^2)$ with array

Algorithm	Time Complexity	Space Complexity	Notes
Kruskal's Algorithm	O(E·log E)	O(V)	Dominated by sorting edges
DFS (Depth-First Search)	O(V+E)	O(V)	V = vertices, E = edges
BFS (Breadth-First Search)	O(V+E)	O(V)	V = vertices, E = edges

Notes on Complexity

- **Dynamic Programming**: Trade space for time by storing subproblem solutions
- **Brute Force**: Exhaustive search guarantees optimal solution but exponential time
- **Sorting**: Comparison-based sorts have O(n log n) lower bound for average/worst case
- **Graph Algorithms**: Complexities depend on graph representation (adjacency list vs matrix)
- **MST Algorithms**: Both Prim's and Kruskal's work correctly with negative edge weights